

React + TypeScript + ASP.NET Core Web API CRUD Tutorial

This tutorial creates:

- ASP.NET Core Web API backend (.NET 8)
 - React + TypeScript frontend (Vite)
 - CRUD operations:
 - Get Jobs
 - Update Job
 - Delete Job
 - EF Core In-Memory database
 - CORS support
-

1. Create Solution Structure

```
mkdir JobPortal
cd JobPortal

mkdir backend
mkdir frontend
```

2. Create Backend

```
cd backend

dotnet new webapi -n JobApi

cd JobApi

dotnet add package Microsoft.EntityFrameworkCore.InMemory
```

3. Create Frontend

Open another terminal:

```
cd JobPortal/frontend

npm create vite@latest job-ui -- --template react-ts

cd job-ui

npm install
```

4. Backend Files

Job.cs

```
public class Job
{
    public int Id { get; set; }

    public string Title { get; set; } = "";

    public string Department { get; set; } = "";

    public decimal Salary { get; set; }
}
```

JobsDbContext.cs

```
using Microsoft.EntityFrameworkCore;

public class JobsDbContext : DbContext
{
    public JobsDbContext(DbContextOptions<JobsDbContext> options)
        : base(options)
    {
    }

    public DbSet<Job> Jobs => Set<Job>();
}
```

JobsController.cs

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;

[ApiController]
[Route("api/[controller]")]
public class JobsController : ControllerBase
{
    private readonly JobsDbContext _db;

    public JobsController(JobsDbContext db)
    {
        _db = db;
    }

    [HttpGet]
    public async Task<IActionResult> GetJobs()
    {
        return Ok(await _db.Jobs.ToListAsync());
    }

    [HttpGet("{id}")]
    public async Task<IActionResult> GetJobById(int id)
    {
        var job = await _db.Jobs.FindAsync(id);

        if (job == null)
            return NotFound();

        return Ok(job);
    }

    [HttpPost]
    public async Task<IActionResult> CreateJob(Job job)
    {
        _db.Jobs.Add(job);

        await _db.SaveChangesAsync();

        return CreatedAtAction(
            nameof(GetJobById),
            new { id = job.Id },
            job
        );
    }
}
```

```

[HttpPut("{id}")]
public async Task<IActionResult> UpdateJob(
    int id,
    Job updatedJob)
{
    var job = await _db.Jobs.FindAsync(id);

    if (job == null)
        return NotFound();

    job.Title = updatedJob.Title;
    job.Department = updatedJob.Department;
    job.Salary = updatedJob.Salary;

    await _db.SaveChangesAsync();

    return NoContent();
}

[HttpDelete("{id}")]
public async Task<IActionResult> DeleteJob(int id)
{
    var job = await _db.Jobs.FindAsync(id);

    if (job == null)
        return NotFound();

    _db.Jobs.Remove(job);

    await _db.SaveChangesAsync();

    return NoContent();
}
}

```

Program.cs

```

using Microsoft.EntityFrameworkCore;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllers();

```

```

builder.Services.AddDbContext<JobsDbContext>(options =>
    options.UseInMemoryDatabase("JobsDb"));

builder.Services.AddOpenApi();

builder.Services.AddCors(options =>
{
    options.AddPolicy("AllowAll", policy =>
    {
        policy.AllowAnyOrigin()
            .AllowAnyMethod()
            .AllowAnyHeader();
    });
});

var app = builder.Build();

app.UseCors("AllowAll");

if (app.Environment.IsDevelopment())
{
    app.MapOpenApi();
}

app.UseHttpsRedirection();

app.MapControllers();

SeedData(app);

app.Run();

static void SeedData(WebApplication app)
{
    using var scope = app.Services.CreateScope();

    var db = scope.ServiceProvider.GetRequiredService<JobsDbContext>();

    if (db.Jobs.Any())
        return;

    db.Jobs.AddRange(
        new Job
        {
            Title = "Software Engineer",
            Department = "Engineering",
            Salary = 80000
        },

```

```

        new Job
        {
            Title = "Data Analyst",
            Department = "Analytics",
            Salary = 60000
        },
        new Job
        {
            Title = "Project Manager",
            Department = "Management",
            Salary = 90000
        });

    db.SaveChanges();
}

```

5. Frontend

App.tsx

```

import { useState } from "react";

interface Job {
    id: number;
    title: string;
    department: string;
    salary: number;
}

const API_URL = "http://localhost:5126/api/jobs";

export default function App() {
    const [jobs, setJobs] = useState<Job[]>([]);
    const [error, setError] = useState<string | null>(null);
    const [loading, setLoading] = useState(false);

    const [editingId, setEditingId] =
        useState<number | null>(null);

    const [draft, setDraft] =
        useState<Job | null>(null);

    async function getJobs() {
        setError(null);
    }

```

```

setLoading(true);

try {
  const resp = await fetch(API_URL);

  if (!resp.ok)
    throw new Error(
      "Req failed: " + resp.status
    );

  setJobs(await resp.json());
}
catch (e) {
  setError(
    e instanceof Error
      ? e.message
      : "Unknown error"
  );
}
finally {
  setLoading(false);
}
}

function startEdit(job: Job) {
  setEditingId(job.id);
  setDraft({ ...job });
}

function cancelEdit() {
  setEditingId(null);
  setDraft(null);
}

async function saveEdit() {
  if (!draft) return;

  try {
    const resp = await fetch(
      `${API_URL}/${draft.id}`,
      {
        method: "PUT",
        headers: {
          "Content-Type":
            "application/json"
        },
        body: JSON.stringify(draft)
      });
  }

```

```

    if (!resp.ok)
      throw new Error(
        "Req failed: " + resp.status
      );

    setJobs(prev =>
      prev.map(j =>
        j.id === draft.id
          ? draft
          : j));

    cancelEdit();
  }
  catch (e) {
    setError(
      e instanceof Error
        ? e.message
        : "Error while saving"
    );
  }
}

async function deleteJob(id: number) {
  try {
    const resp = await fetch(
      `${API_URL}/${id}`,
      {
        method: "DELETE"
      }
    );

    if (!resp.ok)
      throw new Error(
        "Failed: " + resp.status
      );

    setJobs(prev =>
      prev.filter(j =>
        j.id !== id));
  }
  catch (e) {
    setError(
      e instanceof Error
        ? e.message
        : "Delete failed"
    );
  }
}

```



```

return (
  <div
    style={{
      maxWidth: 700,
      margin: "40px auto",
      fontFamily: "sans-serif"
    }}
  >
    <h1>Jobs</h1>

    <button onClick={getJobs}>
      Load Jobs
    </button>

    {loading && <p>Loading...</p>}

    {error &&
      <p style={{ color: "red" }}>
        {error}
      </p>}

    {jobs.length > 0 &&
      <table>
        <thead>
          <tr>
            <th>ID</th>
            <th>Title</th>
            <th>Department</th>
            <th>Salary</th>
            <th>Actions</th>
          </tr>
        </thead>

        <tbody>
          {jobs.map(job =>
            job.id === editingId && draft
              ? (
                <tr key={job.id}>
                  <td>{job.id}</td>

                  <td>
                    <input
                      value={draft.title}
                      onChange={e =>
                        setDraft({
                          ...draft,
                          title:

```

```

        e.target.value
    })
  }
  />
</td>

<td>
  <input
    value={draft.department}
    onChange={e =>
      setDraft({
        ...draft,
        department:
          e.target.value
      })
    }
  />
</td>

<td>
  <input
    type="number"
    value={draft.salary}
    onChange={e =>
      setDraft({
        ...draft,
        salary:
          Number(
            e.target.value
          )
      })
    }
  />
</td>

<td>
  <button
    onClick={saveEdit}
  >
    Save
  </button>

  <button
    onClick={cancelEdit}
  >
    Cancel
  </button>
</td>

```

```

        </tr>
    )
    : (
        <tr key={job.id}>
            <td>{job.id}</td>
            <td>{job.title}</td>
            <td>{job.department}</td>
            <td>{job.salary}</td>

            <td>
                <button
                    onClick={() =>
                        startEdit(job)
                    }
                >
                    Edit
                </button>

                <button
                    onClick={() =>
                        deleteJob(job.id)
                    }
                >
                    Delete
                </button>
            </td>
        </tr>
    )
    )}
</tbody>
</table>
}
</div>
);
}

```

6. Run Backend

```

cd JobPortal/backend/JobApi

dotnet run

```

Example:

```
http://localhost:5126
```

7. Run Frontend

```
cd JobPortal/frontend/job-ui  
  
npm run dev
```

Example:

```
http://localhost:5173
```

Result

You now have:

- ASP.NET Core Web API
- EF Core InMemory database
- React + TypeScript frontend
- GET Jobs
- PUT Update
- DELETE Job
- Inline editing
- CORS support
- OpenAPI support

A complete full-stack CRUD example suitable for interview preparation and demonstrating .NET + React fundamentals.